Cyber Security — Project 1

Password Strength Checker

DecodeLabs Industrial Training Kit — Batch 2026
My walkthrough of how I thought through and built this

1. How I Thought About This Problem

Before writing any code, I asked myself: what actually makes a password weak? Not in a textbook sense practically. Three things came to mind, in order of how much they matter:

First, is it already known to attackers? If a password is on a leaked list like “123456” or “password1”, it doesn't matter how long it is or what characters it uses it's already compromised. So this has to be the very first check, before anything else. This matches the brief's “Gatekeeper Rule”: filter the obviously bad stuff before doing any deeper analysis.

Second, is it long enough? Length is the single biggest factor in how long a brute-force attack takes, because the number of possible combinations grows exponentially with each extra character. A short password is weak no matter how “clever” it looks. I set the bar at 8 characters minimum, since that's the requirement given in the project brief, with a bonus for anything 12+.

Third, does it mix character types? Uppercase, lowercase, digits, symbols. Each one adds more possible characters per position, which again multiplies the number of guesses an attacker would need to try. This is the last check because it's the finer-grained scoring it's what separates “Medium” from “Strong” once a password has already passed the first two gates.

Why check things in this order?

It would be wasteful and misleading to score a leaked password as “Strong” just because it happens to be 12 characters with a symbol in it. So the checks are deliberately sequential, not run all at once and averaged. Fail the leak check → automatically Weak. Fail the length check → automatically Weak. Only if a password clears both of those gates do I bother scoring its character variety. This mirrors how a real security system should reason: cheap, decisive checks first, nuanced scoring second.

2. Building It, Piece by Piece

Step 1 — Checking against known leaked passwords

I kept a small sample set of commonly breached passwords (things like “qwerty”, “admin”, “iloveyou”). In a real production system this would call out to a breach database like Have I Been Pwned, but for this project a local set demonstrates the same logic. The check itself is just a set membership test fast, and it's the reason I used a Python set rather than a list, since checking “is this value in here” is much quicker on a set.

Step 2 — Checking length

This is the simplest check: is len(password) at least 8? Nothing fancy needed here I didn't want to overcomplicate a check that's naturally this straightforward.

Step 3 — Checking character variety

This is where I made a deliberate choice about how to scan the password, not just what to look for. My first instinct was a manual loop walk through each character, set a flag when I find a digit, and so on. But that's more code than necessary, and slower, because Python's built-in string methods are implemented in C under the hood. So instead I used:

```
has_digit = any(char.isdigit() for char in password)
```

The any() function stops scanning the moment it finds one match (“short-circuiting”), so in the best case it doesn't even look at the whole password. I did the same for uppercase, lowercase, and symbols. Four checks, each a single readable line, each as fast as Python allows.

Step 4 — Turning checks into a verdict

Once I had the four boolean flags (has upper / lower / digit / symbol), I added them up into a score from 0–4. Then I made a judgment call: a password only earns “Strong” if it has all four character types and is at least 12 characters I didn't want a technically-varied but short password to be called strong, since length still matters more. A score of 3 or more (but not meeting the Strong bar) becomes “Medium”. Anything else is “Weak”. I also generate specific feedback “add a symbol”, “add an uppercase letter” because a strength label alone doesn't help someone fix their password; telling them exactly what's missing does.

3. The Full Code

Here's the complete program. I've kept the comments in so the reasoning above is visible directly next to the logic it explains.

File: password_strength_checker.py

```
"""
DecodeLabs Industrial Training Kit - Project 1
Password Strength Checker
Track: Defensive Logic | Junior Analyst

Goal:
    Create a program that checks whether a password is weak, medium,
    or strong, based on length, character variety, and known-leaked
    password lists.

Key Skills demonstrated:
    - String handling
    - Conditional logic
    - Security basics (entropy, common-password blocklisting)
"""

import getpass

# A small sample of commonly leaked / breached passwords.
# In a production system this list would be sourced from a
# proper breach database (e.g. Have I Been Pwned's Pwned Passwords API).
COMMON_LEAKED_PASSWORDS = {
    "123456", "123456789", "qwerty", "password", "111111",
    "12345678", "abc123", "1234567", "password1", "12345",
    "iloveyou", "admin", "welcome", "monkey", "letmein",
    "football", "qwerty123", "dragon", "password123", "sunshine",
}

MIN_LENGTH = 8


def check_leaked(password: str) -> bool:
    """Return True if the password appears in the known-leaked list."""
    return password.lower() in COMMON_LEAKED_PASSWORDS


def check_length(password: str) -> bool:
    """Return True if the password meets the minimum length requirement."""
    return len(password) >= MIN_LENGTH


def check_character_variety(password: str) -> dict:
    """
    Scan the password once (O(n)) and report which character
    classes are present using Python's built-in, C-optimized
    string methods rather than manual loops.
    """
    return {
        "has_upper": any(char.isupper() for char in password),
        "has_lower": any(char.islower() for char in password),
        "has_digit": any(char.isdigit() for char in password),
        "has_symbol": any(not char.isalnum() for char in password),
    }


def classify_strength(password: str) -> tuple[str, list[str]]:
```

```
    """
    Classify a password as Weak, Medium, or Strong.

    Returns a tuple of (strength_label, list_of_feedback_notes).
    """
    notes = []

    # --- Gatekeeper checks: automatic fail conditions -----------------
    if check_leaked(password):
        notes.append("This password appears in a known breach/leak list.")
        return "Weak", notes

    if not check_length(password):
        notes.append(f"Too short: needs at least {MIN_LENGTH} characters.")
        return "Weak", notes

    # --- Character variety score --------------------------------------
    variety = check_character_variety(password)
    score = sum(variety.values())  # up to 4: upper, lower, digit, symbol

    if not variety["has_upper"]:
        notes.append("Add at least one uppercase letter.")
    if not variety["has_lower"]:
        notes.append("Add at least one lowercase letter.")
    if not variety["has_digit"]:
        notes.append("Add at least one number.")
    if not variety["has_symbol"]:
        notes.append("Add at least one symbol (e.g. !@#$%).")

    # Bonus: reward extra length beyond the minimum
    length_bonus = len(password) >= 12

    if score == 4 and length_bonus:
        return "Strong", ["Great password!"]
    elif score >= 3:
        return "Medium", notes
    else:
        return "Weak", notes


def display_result(password: str) -> None:
    """Run the checker on a password and print a formatted result."""
    strength, notes = classify_strength(password)

    bar = {"Weak": "[#---------]", "Medium": "[#####-----]", "Strong": "[##########]"}
    print(f"\nPassword Strength: {strength} {bar.get(strength, '')}")
    if notes:
        print("Feedback:")
        for note in notes:
            print(f"  - {note}")


def main():
    print("=" * 50)
    print(" DecodeLabs - Password Strength Checker")
    print("=" * 50)
    try:
        password = getpass.getpass("Enter a password to check: ")
    except Exception:
        # Fallback for environments where getpass isn't supported
        password = input("Enter a password to check: ")

    display_result(password)


if __name__ == "__main__":
```

```
main()
```

4. Testing My Own Logic

Writing the checker is only half the job — I also wanted to convince myself it behaves sensibly. So I ran it against a handful of passwords chosen to hit each category on purpose:

Password	Result	Why
`123456`	Weak	On the leaked-password list — instant fail, length doesn't matter
`password1`	Weak	Also a known leaked password
`Summer2024`	Medium	Passes length + has upper/lower/digit, but no symbol
`Tr0ub4dor!Xz`	Strong	12 chars, has all four character types
`Qx9$mP2vLk#7`	Strong	Same — long and varied

The first two results confirmed the gatekeeper logic works — even though “password1” technically has a digit in it, it never even gets scored on variety, because it's caught by the leak check first. That was the exact behavior I was aiming for.

What the console actually looks like when you run it:

```
================================================
 DecodeLabs - Password Strength Checker
================================================
Enter a password to check:

Password Strength: Medium [#####-----]
Feedback:
  - Add at least one symbol (e.g. !@#$%).
```

5. What I'd Do Next

The brief mentions Project 2 is about hashing and encryption, and that validated input should be passed to hashing — never a raw, unchecked password. This checker is built with that in mind: it's meant to sit right in front of a hashing step, rejecting weak input before it ever reaches Argon2id. If I were to extend this project further, I'd pull the leaked-password list from a real API instead of a small hardcoded set, and add a check for repeated characters or keyboard patterns like “qwerty” or “12345” that aren't in any breach list but are still obviously guessable.